

Supply Chain Security Roadmap

Status: Draft

Current focus: Developers must only pull packages from internally managed package repositories.

Purpose

Bayview's near-term supply chain security roadmap is focused on one control: stop developers from resolving software packages directly from public package repositories such as npm, PyPI, Maven Central, NuGet, RubyGems, Go module proxies, Cargo, public container registries, and similar external sources.

Today, many developers pull packages directly from the internet. If a trusted public package, child dependency, maintainer account, or registry path is compromised, malicious code may execute in developer environments and expose developer secrets. Reactive defenses such as SCA scanning are still useful, but they do not prevent the initial package resolution path from reaching developer machines.

This roadmap narrows the work to package source control. Broader supply chain topics such as sandboxed development, full incident response, container hardening, credential redesign, and general vulnerability remediation can remain in other AppSec workstreams.

Problem Statement

"Implementing control over software development package management resolution, and preventing data infil/exfil."

Currently, we are exposed to software supply chain risk as our developers pull directly from internet package repositories like PyPI and NPM. If we are hit by a "watering-hole attack" such as Shai-Hulud, we would have our developer secrets compromised or full compromise of the application or endpoint. We currently only have reactive defenses that can come in after-the-compromise such as Wiz SCA scanning.

We can solve this by preventing developers from pulling packages from external package repositories, and only allowing them to pull from internally managed package repositories, which observe controls like dependency cooldowns and package scanning.

Requirement

1. Developers must only pull packages from internally managed package repositories.

For this roadmap, an internally managed package repository may be:

- A hosted internal repository for Bayview-owned packages.
- A proxy, mirror, or grouped repository managed by Bayview for approved public packages.
- An approved cleanroom, hardened, or curated package source connected through Bayview-controlled repository management.
- An approved internal container registry or base image repository.

It does not include direct developer access to public package registries or random vendor download URLs unless there is a documented, time-bound exception.